Project 1: Dynamic Programming and Monte Carlo Methods

Taariq Nazar
taariq.nazar@math.su.se

Instructions

Hand in a single typed PDF with your answers, plots and tables, and your code as an appendix. Keep answers short. A plot plus two sentences beats a page of text. See the repository README for how to write the report.

You may use any programming language. Python with NumPy is recommended: the helper code at `github.com/taariqnazar/rl-course-projects` (folder `project-1/helpers`) provides the environment (`gridworld.py`) and plotting functions (`plotting.py`).

The project is worth 100 points and you need at least 50 to pass. The *Grading Criteria* document on the course page explains how this project counts towards your final grade.

A late submission loses 15 points. A failed submission that is resubmitted loses 7 points, plus the 15 points for being late.

Whenever results are random, use **at least 10 random seeds**. Plot the mean with a shaded band of ±1 standard deviation. See Figure 2 for an example.

All exercises use the same environment, so the exact answers from Part A can be used to check your results in Part B.

The environment: a slippery cliff

The agent lives in a 5×5 grid. Cell (r, c) is row r (counted from the **bottom**) and column c (from the left).

- **Start:** $(0, 0)$, marked S.
- **Goal:** $(0, 4)$, marked G. Terminal, reward $+10$ on entry.
- **Cliff:** $(0, 1), (0, 2), (0, 3)$, marked X. Terminal, reward -10 on entry.
- Every other transition gives reward -1.
- **Actions:** $\mathcal{A} = \{\text{up}, \text{right}, \text{down}, \text{left}\}$.
- **Slip:** the chosen action is carried out with probability 0.9. With probability 0.05 each, the agent instead moves in one of the two **perpendicular** directions.
- Moving off the grid leaves the agent where it is, with the usual -1.
- Discount factor $\gamma = 0.9$.

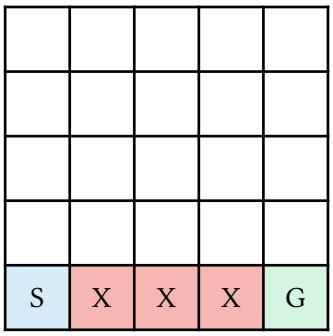


Figure 1: The slippery cliff.

Episodes start in S and end on entering G or X. If an episode reaches $T_{\max} = 200$ steps, stop it there.

Part A: Dynamic Programming (40 p)

In Part A you **know the model**: for every state s and action a you know exactly which next states and rewards are possible and with what probability (Sutton & Barto, Ch. 4).

A1. Build the model (5 p)

Represent the MDP as arrays `P[s, a, s']` (transition probabilities) and `R[s, a]` (expected immediate reward). Terminal states should loop to themselves with reward 0.

Check that every row `P[s, a, :]` sums to 1 and, for non-terminal s, matches the frequencies from calling `env.step(s, a)` many times. Print `P[s, a, :]` for $s = (1, 2)$ and $a =$ down and say where each non-zero entry comes from.

A2. Policy evaluation (10 p)

Let π_{rand} be the uniformly random policy. Throughout Part A, a **sweep** updates every state once, using only the previous sweep's values (keep two arrays). Iterative methods start from $V = 0$ and stop when $\max_s |V_{k+1}(s) - V_k(s)| < 10^{-8}$.

1. Compute $v_{\pi_{\text{rand}}}$ **exactly** by solving the linear system $v = r_\pi + \gamma P_\pi v$. Show it as a 5×5 grid.
2. Compute it again with **iterative policy evaluation** (§4.1). Plot $\max_s |V_k(s) - v_{\pi_{\text{rand}}}(s)|$ against sweep k on a log scale. What shape is the curve, and what sets its slope?
3. Write a function `evaluate(pi)` that returns v_π for any policy, given as an array `pi[s, a]` of action probabilities. **You will reuse it throughout Part B to measure how good your learned policies really are.**

A3. Policy iteration (10 p)

Evaluating a policy tells us how good it is. Acting greedily with respect to its values gives a better one.

Implement policy iteration (§4.3), starting from the policy "always up". Start each evaluation from the previous policy's values (only the first starts from $V = 0$). When improving, keep the current action unless another action is better by more than 10^{-9}.

1. Show the policy as a grid of arrows after each improvement step, and report the optimal v_* as a grid.
2. Count the **total** number of evaluation sweeps over all iterations, and how many sweeps each evaluation took.
3. Does the optimal policy hug the cliff edge? Explain using the slip probability.

A4. Value iteration (15 p)

Value iteration cuts each policy evaluation down to a single sweep.

Implement value iteration (§4.4).

1. Plot $\max_s |V_k(s) - v_*(s)|$ against k. On the same plot, mark the first sweep from which the greedy policy with respect to V_k equals π_* in all non-terminal states and stays that way.
2. Show V_k as heat maps for $k = 1, 2, 4, 8$, and describe how the values spread from G.
3. Compare value iteration with policy iteration by the number of sweeps needed to reach π_*: for value iteration, the sweep marked in part 1; for policy iteration, all evaluation sweeps before the policy first equals π_*.
4. Run value iteration for $\gamma \in \{0.5, 0.9, 0.99\}$. Report the sweeps to convergence in a table and show π_* as arrows for each γ. How does the number of sweeps change? Compare with the bound $\max_s |V_k(s) - v_*(s)| \le \gamma^k \max_s |V_0(s) - v_*(s)|$ and explain any difference. Does the optimal policy change?

Part B: Monte Carlo Methods (52 p)

Without `P`, the agent must learn from sampled transitions (Sutton & Barto, Ch. 5).

> **The rule for Part B:** your learning code may only interact with the environment through
> - `env.reset() -> s`, or `env.reset(s)` to start in a chosen state where the exercise allows it, and
> - `env.step(s, a) -> (s', r, done)`, which **samples** one transition.
>
> It may **not** read `P` or `R`. The only exception is that you may call `evaluate()` from A2 to **measure** a policy the agent has learned.

B1. Returns by hand and by code (5 p)

Consider this episode ($T = 8$):

t	0	1	2	3	4	5	6	7
S_t	(0,0)	(1,0)	(1,1)	(1,0)	(1,1)	(1,2)	(1,3)	(1,4)
A_t	up	right	left	right	right	right	right	down
R_{t+1}	-1	-1	-1	-1	-1	-1	-1	$+10$

1. Compute G_t for every t by hand ($\gamma = 0.9$).
2. Write `returns(rewards, gamma)` using the backward recursion $G_t = R_{t+1} + \gamma G_{t+1}$ and check it against your hand calculation.
3. Give the first-visit and every-visit MC estimates of $V(1, 0)$ and $V(1, 1)$ from this single episode. Why do they differ?

B2. Monte Carlo prediction (10 p)

Estimate $v_{\pi_{\text{rand}}}$ with first-visit and every-visit MC prediction (§5.1), with all episodes starting in S and $V \equiv 0$ until a state is first visited.

1. Plot the RMSE between V and $v_{\pi_{\text{rand}}}$ over non-terminal states against the number of episodes (up to 2000) for both variants.
2. Show the per-state absolute error after 2000 episodes as a heat map. Which states are worst, and why?

B3. Greedy MC control: what goes wrong (10 p)

Improving a policy from V alone requires the model, so we estimate action values $Q(s, a)$ instead.

Implement first-visit MC control where the agent **always acts greedily** with respect to Q, breaking ties at random. Start every episode in S with $Q \equiv 0$.

1. Every 500 episodes, take the greedy policy π_k and record $v_{\pi_k}(\mathrm{S})$ with `evaluate()`. Plot it against episodes (up to 10000) together with $v_*(\mathrm{S})$.
2. After training, show the learned policy as arrows and a heat map of the visit counts $N(s, a)$, summed over actions or shown per action.
3. Explain what happened. Point to specific (s, a) pairs in your counts. Would more episodes help?

B4. Two ways to explore (15 p)

The greedy agent needs a source of exploration.

1. **Exploring starts** (§5.3): start each episode in a uniformly random non-terminal state with a uniformly random first action, then act greedily. Record $v_{\pi_k}(\mathrm{S})$ as in B3.

2. **On-policy ε-greedy** (§5.4): start in S and act ε-greedily for $\varepsilon \in \{0.05, 0.1, 0.2\}$. Record $v_{\pi_k}(\mathrm{S})$ as in B3 for each ε, where π_k is the **greedy** policy derived from Q.
3. After training, evaluate the ε-greedy policy the agent **actually follows** for each ε, and report it in a table next to $v_*(\mathrm{S})$. Explain the gap.
4. Plot greedy, exploring starts and the three ε-greedy curves in one figure and rank them. How does the choice of ε trade learning speed against final performance?

B5. Off-policy prediction with importance sampling (12 p)

ε-greedy keeps exploring forever. Off-policy methods learn about one policy while following another.

The target policy is the deterministic π_* from Part A. Generate episodes from S with a **behaviour policy** b that is ε_b-greedy around π_*, and estimate $v_*(\mathrm{S})$ from those episodes alone, using the return from $t = 0$ of each episode (§5.5):

- **ordinary** importance sampling, and
- **weighted** importance sampling (the estimate is 0 until the first non-zero ratio).

1. For $\varepsilon_b \in \{0.5, 1.0\}$ ($\varepsilon_b = 1$ is uniformly random behaviour), plot both estimates against episodes (up to 5000, log x-axis) for 20 seeds, with **each seed as its own thin line**, plus the true value.
2. Report the mean and standard deviation over seeds of each estimator after 10, 100, 1000 and 5000 episodes.
3. Most importance ratios ρ are exactly 0. Why? What do the rare non-zero ones look like when $\varepsilon_b = 1$?
4. Which estimator would you prefer, and how does the answer depend on ε_b?

Part C: Putting it together (8 p)

Fill in the table below **from your own experiments** and add a few sentences comparing the methods.

Method	Needs model?	How it explores	Main weakness	Converges to
Policy iteration				
Value iteration				
MC, greedy				
MC, exploring starts				
MC, ε-greedy				
MC, off-policy prediction (OIS / WIS)				

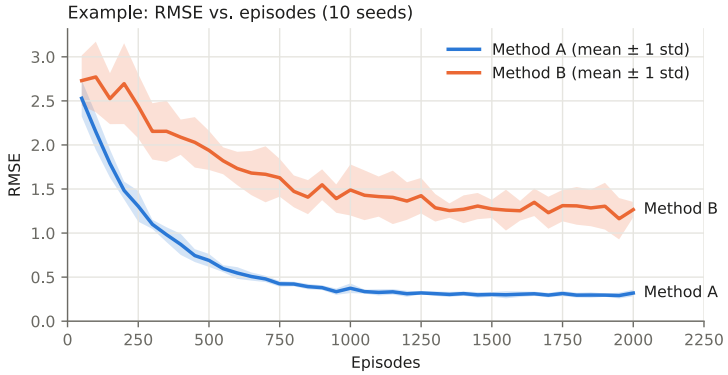


Figure 2: Example: mean over 10 seeds with a ±1 standard deviation band. Synthetic data.